OrgMatcher – Honors Contract Project

Lance Joseph A. Trasporto

College of Engineering, University of North Texas

Honors Contract

Dr. Xinrui Cui

February 11, 2026 **(FIXME)**

I. Introduction

As of January 2026, there are over 400 active organizations at UNT. For incoming freshmen or transfer students, this could be quite intimidating, even if there are many choices presented before them. How could they find organizations that they believe they can fit into? How could they be sure these organizations align with their general interests and hobbies? When a student visits UNT's OrgSync page, they are provided with a search bar if they already have an organization in mind, as well as a "filter categories" section to look for organizations that align with their general interests. However, what about students who do not have a clear idea of what they want?

OrgMatcher is for incoming freshmen and transfer students at the University of North Texas who feel overwhelmed by the vast number of campus organizations. OrgMatcher is a personalized, natural language processing (NLP)-driven discovery engine that uses advanced text-vectorization to bridge the gap between a student's self-described passions and the missions of active campus groups, unlike the traditional OrgSync search bar, which requires specific keyword knowledge, by providing an intent-based interface that simplifies the path to campus involvement and fosters immediate belonging.

This paper will cover the description, the program's development process, methodologies used with emphasis on natural language processing techniques, the vision behind the program, and a discussion section on how the application can be considered a scholarly contribution and improved upon.

II. Description

OrgMatcher is a Natural Language Processing (NLP) program designed to match students with suitable campus organizations. It prompts the user for a string of text outlining the qualities they look for in an organization, such as hobbies, interests, social connections, personal growth, and other personal factors. The program uses scraped data from over 400 UNT organizations, including their name, acronym, summary, and description. By matching the terms in the user's input against the content of each organization's contents from the UNT OrgSync page, the

program returns the five most relevant organizations based on compatibility using NLP methods such as TF-IDF, lemmatization, and cosine similarity.

III. Process and Methodology

For this program to function, it first needs data to compare the user input with. This is done using the Python programming language, as well as a well-known library called BeautifulSoup. I created a script that automatically scrapes the links of all organizations through UNT's OrgSync API. These links are then stored in a .csv (comma-separated value) file. With this CSV file in hand, another script automatically accesses the links in the file line by line, parsing and scraping an organization's name, acronym (if available), summary, description, and logo.

The parsed, raw HTML data is then cleaned and stripped of any commas, links, colons, or semicolons, and is passed into a SQLite database for easier access to club information that will be used later in the program. From now on, we will refer to the data stored in the database as the corpus. A **corpus** is a collection of text that is used to study language use or train AI models (Egger & Gokce, 2022). In this scenario, we will use the corpus to analyze the term frequency and inverse document frequency for each word in the document.

Then begins the process of vectorization and TF-IDF, where Python libraries scikit-learn and pandas will be used to implement the program's natural language processing model. This model will vectorize the user's input–a string of text–into a list of words that will then be used, compared, and given a "score" or weight through TF-IDF. Parameters such as the organization's name, short name (e.g., IEEE for Institute of Electrical and Electronics Engineers), summary, and description will also be combined and vectorized. From now on, we will refer to these combined parameters as the document. Additionally, we will give the "short name" inputs extra weight, since we assume that if a student includes an organization's acronym in an input, then there's already an initial interest. For this project, let us define a document with this equation:

document = organization's name + short name*2 + summary + description

Term frequency-inverse document frequency (TF-IDF) is a widely used natural language processing technique that calculates a numerical weight for each word to reflect how important that word is to a specific document within a collection or corpus (Robertson, 2004). TF-IDF is actually a combination of two equations: Term frequency (TF) and Inverse Document Frequency (IDF). These two equations are multiplied to give a word a score or weight that determines a word's importance.

Term frequency measures how often a word appears in a document. It is defined with the following general equation:

$$TF(t,d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

The equation takes parameters *t* and *d*, where *t* is the number of times a word appears in document *d*. For this example, we use the organization Asian Student Association. As mentioned above, the document consists of the organization's name, short name, summary, and description. It is also good to note that Python's scikit-learn library converts all words to lowercase, strips all punctuation, and disregards single-letter words such as 'I' and 'a' during vectorization, giving us the following:

['asian', 'student', 'association', 'asa', 'group', 'striving', 'to', 'promote', 'asian', 'culture', 'through', 'informative', 'meetings', 'intramural', 'sports', 'and', …]

Using this information, let us apply the word 'Asian' to the text frequency formula using the vectorized document above. For context, the word 'Asian' appears seven times, and the word 'Asians' appears once. The program will count these words similarly by using a process called lemmatization, making the base word 'Asian' appear eight times. **Lemmatization** is a technique that reduces a word to its base or root form (Pant et al., 2024). For example, words such as 'runs', 'running', and 'ran' will all be reduced to 'run'. In this scenario, 'Asians' is reduced to 'Asian'. We get the following result:

$$TF = \frac{\text{Number of times 'Asian' appears in document}}{\text{Total number of terms in document}} = \frac{8}{86} \approx 0.0930$$

The term frequency formula thus gives the word ‘Asian’ a score of around 0.0930.

On the other hand, inverse document frequency measures how rare a given word is across all 400+ documents. It is defined with the following general equation:

$$IDF(t, D) = \log_e \left(\frac{\text{Total number of documents in corpus}}{1 + \text{Number of documents with term } t \text{ in them}} \right) = \log_e \left(\frac{D}{1 + df(t)} \right)$$

The equation takes parameters *t* and *D*, where *t* is the term, and *D* is the corpus. The variable *df* is the term’s document frequency. **Document frequency** measures the number of unique documents that contain a specified term (Joho & Sanderson, 2007).

Let us again use the word ‘Asian’ as an example. The term ‘Asian’ has a document frequency of 8, meaning across the whole corpus of 416 documents, the term ‘Asian’ appeared in 8 of them. Plugging this into our equation, we get:

$$IDF = \log_e(\frac{416}{1+8}) \approx 3.8335$$

The inverse document frequency thus gives the term ‘Asian’ a score of around 3.8335. The combined TF-IDF weight is obtained by multiplying the results from the TF and the IDF equations.

$$TF - IDF(t, d, D) = TF(t, d) \cdot IDF(t, D) = 0.0930 \cdot 3.8335 \approx 0.3565$$

For comparison, let us apply the same logic using a more common word, such as ‘the’. The word ‘the’ appears on 371 out of the 416 documents in the corpus. Below, variable *d* is the document for the Asian Student Association, and *D* is the corpus.

$$TF(\text{'the'}, d) = \frac{3}{86} \approx 0.0349$$
$$IDF(\text{'the'}, D) = \log_e \left(\frac{416}{1 + 371} \right) \approx 0.1118$$
$$TF - IDF(\text{'the'}, d, D) \approx 0.0039$$

The word 'Asian' weighs around 0.3565, while the word 'the' weighs around 0.0039. This reveals that the word 'Asian' is approximately 91 times more significant in the corpus, meaning the program will assign it a higher value than common words like 'the' in a student's input. Since the word 'the' is commonly occurring and not context-dependent, we will add it to a list of stop words and disregard it. **Stop words** are common words (such as 'the', 'is', 'of') that have little semantic meaning and are filtered out for a more accurate analysis of terms (Wilbur & Sirotkin, 1992).

Then, the student's input will be compared with the documents we used TF-IDF on using cosine similarity. **Cosine similarity** is a commonly used technique in natural language processing that measures similarity between text documents based on their term vectors (Rahutomo et al., 2012). Two completely similar vectors will have a cosine similarity of 1, two completely dissimilar vectors will have a cosine similarity of 0, and vectors with some similar terms will fall between 0 and 1. The closer the number is to 1, the higher the similarity. It is defined with the following general equation:

$$\text{Cosine Similarity} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2} \sqrt{\sum_{i=1}^{n} B_i^2}}$$

Where variables A and B are vectors of terms and A_i and B_i are elements at the ith position, i being the first or initial word in the vector, and n being the length of the vector.
For a quick example, let us consider the following sentences. Remember that single-letter words and punctuation will be stripped, and all words will be converted to lowercase.

A. "UNT is a great university for students!" → ['unt', 'great', 'university', 'students']
B. "I love UNT and the campus." → ['love', 'unt', 'campus']
C. "Hi there." → ['hi', 'there']

D. “The UNT campus is great, and I love it!” → [‘unt’, ‘campus’, ‘great’, ‘love’]

With this information, let’s create a frequency table for the terms in the vector and remove the stop words. The second row will be for sentence A, the third row for sentence B, and so forth.

‘unt’	‘great’	‘university’	‘students’	‘campus’	‘love’	‘hi’	‘there’
1	1	1	1	0	0	0	0
1	0	0	0	1	1	0	0
0	0	0	0	0	0	1	1
1	1	0	0	1	1	0	0

Now, we can substitute these numbers directly into the formula.
Using the formula on sentences A and B, we get:

$$\text{Cosine Similarity}(A, B) = \frac{(1 \cdot 1) + (1 \cdot 0) + (1 \cdot 0) + (1 \cdot 0) + (0 \cdot 1) + (0 \cdot 1) + (0 \cdot 0) + (0 \cdot 0)}{\sqrt{1^2 + 1^2 + 1^2 + 1^2 + 0^2 + 0^2 + 0^2 + 0^2}\sqrt{1^2 + 0^2 + 0^2 + 0^2 + 1^2 + 1^2 + 0^2 + 0^2}} \approx 0.2887$$

Using the formula on sentences A and C, we get:

$$\text{Cosine Similarity}(A, C) = \frac{(1 \cdot 0) + (1 \cdot 0) + (1 \cdot 0) + (1 \cdot 0) + (0 \cdot 0) + (0 \cdot 0) + (0 \cdot 1) + (0 \cdot 1)}{\sqrt{1^2 + 1^2 + 1^2 + 1^2 + 0^2 + 0^2 + 0^2 + 0^2}\sqrt{0^2 + 0^2 + 0^2 + 0^2 + 0^2 + 0^2 + 1^2 + 1^2}} = 0$$

Using the formula on sentences B and D, we get:

$$\text{Cosine Similarity}(B, D) = \frac{(1 \cdot 1) + (0 \cdot 1) + (0 \cdot 0) + (0 \cdot 0) + (1 \cdot 1) + (1 \cdot 1) + (0 \cdot 0) + (0 \cdot 0)}{\sqrt{1^2 + 0^2 + 0^2 + 0^2 + 1^2 + 1^2 + 0^2 + 0^2}\sqrt{1^2 + 1^2 + 0^2 + 0^2 + 1^2 + 1^2 + 0^2 + 0^2}} \approx 0.8660$$

As observed, sentences B and D are most similar, with a cosine similarity of around 0.8660, which is evident based on the frequency table above. On the other hand, we can observe that sentences A and C are completely dissimilar.

In the actual program, this process will be enacted on all 416 documents in the corpus, as well as for the user's input string. The TF-IDF and cosine similarity scores will be multiplied, and the program will return and display to the user the first 5 organizations with the highest scores and the greatest relevance based on the user's input.

Lastly, the program is given a user-friendly interface by utilizing the frameworks React for the frontend and FastAPI for the backend (**FIXME**)

IV. Vision

The vision for this program is to help freshmen and transfer students have a smoother transition into UNT by connecting them with relevant campus organizations, resulting in a stronger sense of community and belonging from the outset. I personally felt overwhelmed by the number of organizations here at UNT, and I believe OrgMatcher can effectively narrow down a student's choices for involvement.

I also believe that a program like OrgMatcher would be beneficial to UNT if it were integrated into the OrgSync website. A simple addition, such as an alternative text box that prompts a student for their interests and hobbies, would increase student exposure to the diverse range of organizations here at UNT. Furthermore, the application's scope could be expanded beyond just organizations to include upcoming campus events, meetings, career fairs, and other happenings.

V. Discussion

Compared to the current OrgSync website, which uses a simple search bar and a category filter, OrgMatcher would be a great addition for matching students to organizations they are personally interested in. As previously mentioned, students must know what they are searching for when using a traditional search bar. In contrast, this NLP application will compare the student's interests with the description listed on the organization's website.

A limitation of this program is its inability to parse an organization's category. Based on the UNT OrgSync website's source code, all organization category headers appear to be empty.

Manually adding categories for each organization is an option, but it would be highly impractical and require continuous manual updates as new organizations are created or updated.

Another limitation of this program is that TF-IDF does not account for context or human intentions. This approach could be handled using transformers that are specifically trained to learn about the 400+ organizations at UNT. However, the current implementation is sufficiently effective for the project's scope.

A valuable future addition would be implementing sentiment analysis alongside the existing TF-IDF model. This could increase matching accuracy by gauging the organization's tone or atmosphere based on the listed description.

Finally, OrgMatcher is asserted as a scholarly contribution. It serves as a testament to the advancement of AI and an example of its real-world applications, using industry-standard natural language processing methods to solve practical social problems students might face during their time at UNT and connecting them to organizations where they can build community and thrive in a new environment.

References

Egger, Roman & Gokce, Enes. (2022). Natural Language Processing (NLP): An Introduction: Making Sense of Textual Data. 10.1007/978-3-030-88389-8_15.

Joho, Hideo & Sanderson, Mark. (2007). Document Frequency and Term Specificity.

Pant, Vinay & Sharma, Rupak & Kundu, Shakti. (2024). An overview of Stemming and Lemmatization Techniques. 10.1201/9781003430421-31.

Rahutomo, Faisal & Kitasuka, Teruaki & Aritsugi, Masayoshi. (2012). Semantic Cosine Similarity.

Robertson, Stephen. (2004). Understanding Inverse Document Frequency: On Theoretical Arguments for IDF. Journal of Documentation - J DOC. 60. 503-520. 10.1108/00220410410560582.

Wilbur, W. & Sirotkin, Karl. (1992). The automatic identification of stop words. Journal of Information Science. 18. 45-55. 10.1177/016555159201800106.